

СОФИЙСКИ УНИВЕРСИТЕТ
„СВ. КЛИМЕНТ ОХРИДСКИ“



ФАКУЛТЕТ ПО МАТЕМАТИКА
И ИНФОРМАТИКА

ДЪРЖАВЕН ИЗПИТ

ЗА ПОЛУЧАВАНЕ НА ОКС „БАКАЛАВЪР ПО СОФТУЕРНО ИНЖЕНЕРСТВО“

ЧАСТ I (ПРАКТИЧЕСКИ ЗАДАЧИ)

Драги абсолвенти:

- Попълнете факултетния си номер в горния десен ъгъл на всички листове.
- Пишете само на предоставените листове, без да ги разкопчавате.
- Решението на една задача трябва да бъде на същия лист, на който е и нейното условие (т.е. може да пишете отпред и отзад на листа със задачата, но не и на лист на друга задача).
- Ако имате нужда от допълнителен лист, можете да поискате от квесторите.
- На един лист не може да има едновременно и чернова, и белова.
- Черновите трябва да се маркират, като най-отгоре на листа напишете „ЧЕРНОВА“.
- Ако решението на една задача не се побира на нейния лист, трябва да поискате нов бял лист от квесторите. Той трябва да се защити с телбод към листа със задачата.
- Всеки от допълнителните листове (белова или чернова) трябва да се надпише най-отгоре с вашия факултетен номер.
- Черновите също се предават и се защитават в края на работата.
- Времето за работа по изпита е 3 часа.

Изпитната комисия ви пожелава успешна работа!

Задача 1. Задачата да се реши на езика C++. Реализирайте дадените по-долу функции и отговорете на въпросите. Ако е нужно, можете да дефинирате други, помощни функции.

А) `bool resize(int*& arr, size_t size, size_t new_size)`, която получава указател `arr` към масив съдържащ `size` елемента.

- Ако `size==new_size`, функцията не прави нищо и връща `true`.
- В противен случай заделя (с `new`) нов масив с размер `new_size` и започва последователно да копира в него всички елементи от `arr`, но не повече от `new_size` на брой. Накрая функцията освобождава (с `delete`) стария масив и връща през `arr` адреса на новия. Като резултат връща `true`. Забележете, че този случай се прилага и когато `arr==nullptr` или `size==0`.
- Ако възникне грешка при заделянето на памет, функцията да връща `false` и да НЕ освобождава оригиналния масив. В този случай `arr` не се променя.
- Ако `new_size > size`, новите елементи да се оставят неинициализирани.
- В частност, ако `new_size==0`, функцията освобождава масива, записва `nullptr` в `arr` и връща `true`.

Б) `bool contains(int* arr, size_t size, int value)`, която получава указател `arr` към сортиран възходящо масив, съдържащ `size` елемента.

- Функцията връща `true`, ако стойността `value` се съдържа в масива. В противен случай да се връща `false`.
- Функцията да бъде рекурсивна и да използва алгоритъма за двоично търсене (`binary search`).
- Решения, които са итеративни, реализират друг алгоритъм или използват глобални променливи, се оценяват с нула точки.

В) Опишете, стъпка по стъпка, какъв е коректният начин да се сравнят две стойности `A` и `B` от тип `double`, за да се провери дали те са равни или за да се определи коя е по-малка. Обосновете отговора си – обяснете защо сравнението на такива стойности е по-особено и как описаните от вас стъпки решават проблема. Реализирайте следните `inline` булеви функции:

- `equal`, която проверява дали две такива стойности са равни;
 - `less`, която проверява дали дадена стойност `A` е по-малка от друга – `B`.
-

Примерно решение

А) Една възможна реализация на функцията `resize` е следната:

```
bool resize(int*& arr, size_t size, size_t new_size)
{
    if(size == new_size)
        return true;

    int* temp = nullptr;

    if(new_size > 0) {
        try {
            temp = new int[new_size];
        }
        catch(std::bad_alloc&) {
            return false;
        }
    }

    size_t limit = std::min(size, new_size);

    for(size_t i = 0; i < limit; ++i)
        temp[i] = arr[i];

    delete [] arr;
    arr = temp;

    return true;
}
```

Б) Една възможна реализация на функцията `contains` е следната:

```
bool contains(int* arr, size_t size, int value)
{
    if (!arr || size == 0)
        return false;

    size_t mid = size / 2;

    if(arr[mid] == value)
        return true;
    else if(arr[mid] < value)
        return contains(arr+mid+1, size-mid-1, value);
    else
        return contains(arr, mid, value);
}
```

В) Възможен отговор: Числата с плаваща запетая не винаги могат да се представят точно в двоична форма. Понякога това води до малки грешки при изчисленията и директното сравнение на две стойности от тип `double` може да доведе до неочаквани резултати. Например, две стойности, които би трябвало да са равни, могат да се различават с много малка стойност поради грешки при представянето. Също, в зависимост от използваното от компилатора представяне, възможно е да се поддържат стойности

от тип NaN и Inf, които да изискват особено внимание. Например, ако искаме две стойности от тип NaN да се считат за еквивалентни, може да се наложи това да се разпише изрично, защото по правило сравнение на NaN с друга стойност е false, дори когато сравняваме NaN с NaN.

Възможен подход:

1. За сравнение на числа определяме число `epsilon` – допустимата грешка при сравнение. То трябва да бъде достатъчно малко, за да не влияе значително на резултата от сравнението.
2. За да проверим дали две стойности A и B са равни, изчисляваме абсолютната разлика между тях и я сравняваме с `epsilon`. Ако разликата е по-малка от `epsilon`, считаме, че A и B са равни.
3. За да проверим дали A е по-малко от B, първо проверяваме дали A и B са равни (използвайки горната функция). Ако не са равни, тогава директно сравняваме A и B.
4. Ако е нужно, обработваме отделно NaN и Inf.

```
bool equal(double A, double B, double epsilon = 1e-10) {  
    if(std::isnan(A) && std::isnan(B))  
        return true;  
    else if(std::isinf(A) && std::isinf(B))  
        return A == B;  
    else  
        return std::abs(A - B) < epsilon;  
}  
  
bool less(double A, double B, double epsilon = 1e-10) {  
    return !equal(A, B) && (A < B);  
}
```

Критерии за оценяване

Сумата от точките от подусловия А), Б) и В) се закръглява до цяло число.

А)

- 1 точка за коректно поведение при `size==new_size`
- 1 точка за коректно заделяне на памет, вкл. проверка дали заделянето е успешно и предприемане на действия при грешка.
- 1 точка за коректно копиране, което не излиза от границите на двата масива.
- 1 точка за коректно освобождаване на паметта.
- 1 точка за коректно връщане на стойност през `arr`.
- 1 точка за коректно поведение когато `size==0` или `arr==nullptr`.
- Точките се намаляват при наличие на проблеми в реализацията.

Б)

- Ако решението не е рекурсивно, използва глобални променливи, или друг алгоритъм – 0 точки
- 2 точки за коректно описани гранични случаи (дъно на рекурсията).
- 2 точки ако коректно са направени рекурсивните обръщения.
- Точките се намаляват при наличие на проблеми в реализацията.

В)

- 0,5 точки за коректно описание на стъпките.
- 0,5 точки за коректна обосновка/обяснение.
- 0,5 точки за напълно коректна реализация на `equal`.
- 0,5 точки за напълно коректна реализация на `less`.
- Ако не е споменато нищо относно NaN и Inf, точките се намаляват с 0,25.

Задача 2. *Облачна софтуерна система за съхранение и споделяне на файлове* има следните изисквания:

- R1. Потребителите трябва да имат възможност да качват, изтеглят и изтриват файлове.
- R2. Всеки потребител трябва да има лична папка за неговите файлове. Също така има и възможност да споделя папки с други потребители на системата.
- R3. Системата трябва да поддържа история на промените на файловете (стари версии на файловете), така че потребителите да могат да се връщат към предишни състояния на файл.
- R4. При промени по даден файл от потребител, трябва да се изпрати съобщение на всички други потребители, с които файлът е споделен.
- R5. Системата трябва да позволява търсене на файлове по различни критерии, например име, тип, съдържание и т.н.
- R6. Системата трябва да поддържа едновременен достъп от множество устройства за един и същ потребител.
- R7. Системата трябва да бъде достъпна както чрез уеб интерфейс, така и чрез мобилно приложение.
- R8. Системата трябва да осигурява надеждно съхранение на данните – да няма загуба на данни при срыв.
- R9. С цел сигурност, системата трябва да криптира данните както при изпращане, така и при съхранение.
- R10. При пиково нарастване на броя на потребителите (например 100000 в рамките на 2 секунди), системата да осигурява отговор за ≤ 2 секунди за 95% от заявките.

Задание:

- 1. Да се предложи архитектурен стил (или комбинация от няколко), който би могъл да удовлетвори изискванията на системата, и да се обоснове защо този избор е подходящ спрямо нуждите на системата относно мащабируемост (*scalability*), наличност (*availability*), производителност (*performance*) и сигурност (*security*).
- 2. Да се опишат възможните недостатъци, като например компромиси с други качествени изисквания и/или ограничения, които този избор би породил.
- 3. Да се създаде графично представяне на системата (*Декомпозиция на Модулите*) в съответствие с избрани(те) от Вас архитектурен(и) стил(ове), заедно с кратко описание за всеки модул. *Диаграми без описание не носят точки.*

Примерно решение

Забележка: Решението е ориентировъчно. Отговорите може да се формулират по различен начин и да донесат пълни точки, стига да са верни.

Подусловие 1 (4 точки)

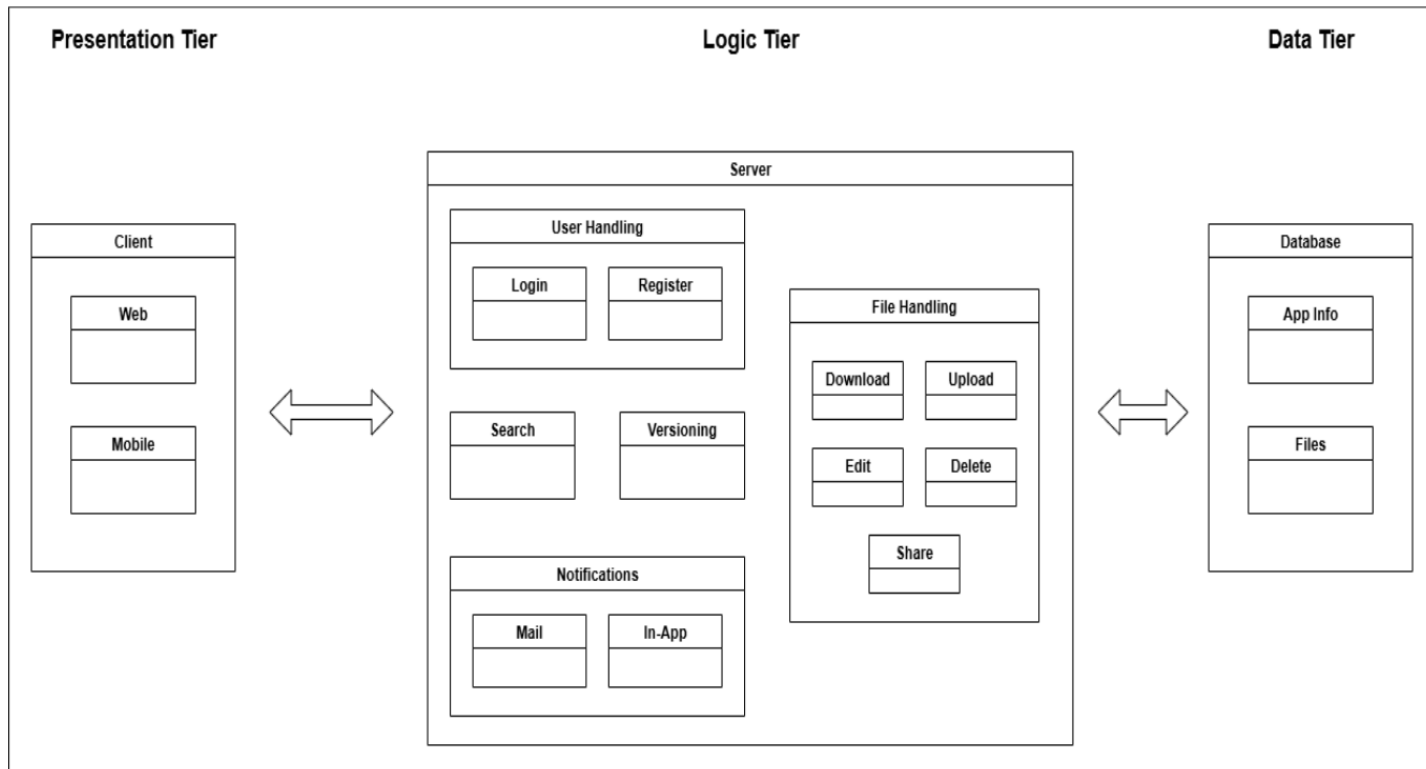
Архитектурен стил: *Многослойна архитектура (N-tier architecture)* Многослойната архитектура (*N-tier architecture*) е архитектурен стил, при който системата е разделена на логически слоеве (напр. нива за представяне, логика, данни), всеки със специфична отговорност. Чрез това разделяне имаме възможността да променяме специфично ниво, вместо да преработваме цялото приложение. Често са на отделни физически машини, комуникирайки чрез добре дефинирани интерфейси.

- **Мащабируемост:** Многослойната архитектура позволява независимо скалиране на отделните нива, за да се справят с различни нива на натоварване – напр. множество инстанции на приложния слой с load balancer.
- **Наличност:** Многослойната архитектура дава възможност за прилагане на различни тактики за повишаване на наличността във всеки логически слой, като например репликация на базата данни и балансиране на натоварването.
- **Производителност:** Логическото разделение и изолацията между отделните слоеве улеснява прилагането на различни тактики за всеки слой (напр. кеширане, асинхронна обработка) за оптимизиране и по-ефективно управление на ресурсите.
- **Сигурност:** На всяко ниво може да бъде внедрена политика за сигурност, което подобрява цялостната сигурност – напр. комуникационни протоколи в презентационния слой, контрол на достъп в приложния слой, криптиране на данните в слоя за съхранение.

Подусловие 2 (2 точки)

Компромиси и ограничения:

- **Комплексност:** При многослойната архитектура дефинирането, проектирането и управлението на множество отделни нива може да увеличи комплексността на приложението.
- **Латентност:** Непрекъснатата комуникация между отделните слоеве на приложението може да повлияе значително на производителността.
- **Отстраняване на грешки:** Тъй като всяка заявка преминава през няколко слоеве, може да е по-трудно да се открие проблема.
- **Дублиране на логика:** Има вероятност една и съща бизнес логика да се имплементира в множество слоеве, затруднявайки поддръжката.

Подусловие 3 (4 точки)**Графично представяне на системата**

На фигурата е представена трислойната (3-tier) архитектура на приложението – Презентационен слой (*Presentation tier*), Логически слой (*Logic tier*) и Слой за съхранение на данни (*Data tier*).

Първият от тях, Client модула, отговаря за взаимодействието между потребителя и системата. Неговите подмодули, Web и Mobile, се грижат съответно за достъп чрез уеб базиран интерфейс и мобилно приложение.

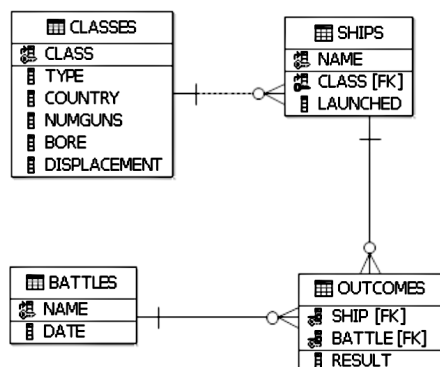
В Логическия (Logic) слой, се намира бизнес логиката на системата. Подмодулът User Handling се грижи за достъпа на потребителите. Съставен е от два подмодула, Register и Login, чрез които се манипулират потребителските профили (регистрация и удостоверяване и оторизация). Подмодулът File Handling предоставя основната функционалност на системата – чрез неговите модули се осъществяват действията за качване, изтегляне, изтриване, модифициране и споделяне на файл. След промяна на файл, подмодулът Notifications, се грижи за известяването на потребителите с които той е бил споделен, чрез мейл (подмодул Mail) или чрез вътрешни за системата нотификации (подмодул In-App). Останалите два подмодула, Search и Versioning, се грижат съответно за предоставянето на удобен интерфейс за търсене и запазването на предишни версии на файловете.

В последния слой, модулът Database, се грижи за съхранението на данните: в подмодула App Info се пази информация за потребители, файлове, версии, сесии и т.н. и в подмодула Files се съхранява реалното съдържание на файловете.

Критерии за оценяване

- Подусловие 1 – 4 точки
- Подусловие 2 – 3 точки
- Подусловие 3 – 5 точки

Задача 3. Дадена е базата от данни Ships, в която се съхранява информация за кораби (Ships) и тяхното участие в битки (Battles) по време на Втората световна война. Всеки кораб е построен по определен стереотип, определящ класа на кораба (Classes).



Таблицата Classes съдържа информация за класовете кораби:

- class — име на класа, първичен ключ;
- type — тип: 'bb' за бойни кораби и 'bc' за бойни крайцери;
- country — държавата, която строи такива кораби;
- numGuns — броят на основните оръдия;
- bore — калибърът им (диаметърът на отвора на оръдието в инчове);

• displacement — водоизместимост в тонове.
Таблицата Ships съдържа информация за корабите:

- name — име на кораб, първичен ключ;
- class — име на неговия клас;
- launched — годината, в която корабът е пуснат на вода.

Таблицата Battles съдържа информация за битките:

- name — име на битката, първичен ключ;
- date — дата на провеждане.

Таблицата Outcomes съдържа информация за резултата от участието на даден кораб в дадена битка, като колоните ship и battle заедно формират първичния ключ:

- ship — име на кораба;
- battle — име на битката;
- result — резултат от битката: 'sunk' за потънал, 'damaged' за повреден и 'ok' за победил.

1) Да се напише заявка, която показва имената на класовете кораби, които никога не са били потопени.

2) Да се попълнят празните места в следната заявка така, че да се изведе името на кораба, държавата му и името на битката, ако резултатът от участието е 'ok':

```
SELECT T1.ship, T3.country, T1._____  
FROM Outcomes AS T1  
JOIN _____ AS T2 ON T1.ship = T2.name  
JOIN Classes AS T3 ON T2.class = T3.class  
WHERE T1._____ = 'ok';
```


Примерно решение на подзадача 1

Възможно решение: имената на класовете кораби от таблицата `Classes`, за които няма съответствие в колоната `class` на таблицата `Ships`, съединена с таблицата `Outcomes` по първичния си ключове и филтрирана по стойност `'sunk'` на колоната `result`, т.е. няма кораб от този клас, който е бил потопен в битка, се получават от следната заявка:

```
SELECT DISTINCT c.class
FROM Classes c
WHERE c.class NOT IN (
    SELECT s.class
    FROM Ships s
    JOIN Outcomes o ON s.name = o.ship
    WHERE o.result = 'sunk'
);
```

Примерно решение на подзадача 2

По ред на празните слотове:

- a) battle
- б) Ships
- в) result

Критерии за оценяване

- 1) Общо 6 т., като за всяка сгрешена клауза броят на точките се намалява с 2 до достигане на 0 т.
 - 2) Общо 6 т., по 2 т. на коректно посочен верен отговор.
-

Задача 4.

Даден е следният псевдокод:

```
1  Program LoanEligibilityCheck
2
3  ApplicantAge: Integer;
4  ApplicantIncome: Integer;
5  ApplicantCreditScore: Integer;
6
7  Begin
8
9  Read(ApplicantAge)
10 Read(ApplicantIncome)
11 Read(ApplicantCreditScore)
12
13 If ApplicantAge < 21 OR ApplicantAge > 60
14 Then
15     Print("Applicant not eligible due to age")
16 Else
17     If ApplicantIncome < 30000
18     Then
19         Print("Applicant not eligible due to income")
20     Else
21         If ApplicantCreditScore < 650 AND ApplicantIncome < 50000
22         Then
23             Print("Applicant not eligible due to low credit score and income")
24         Else
25             Print("Applicant eligible for loan")
26         Endif
27     Endif
28 Endif
29
30 End
```

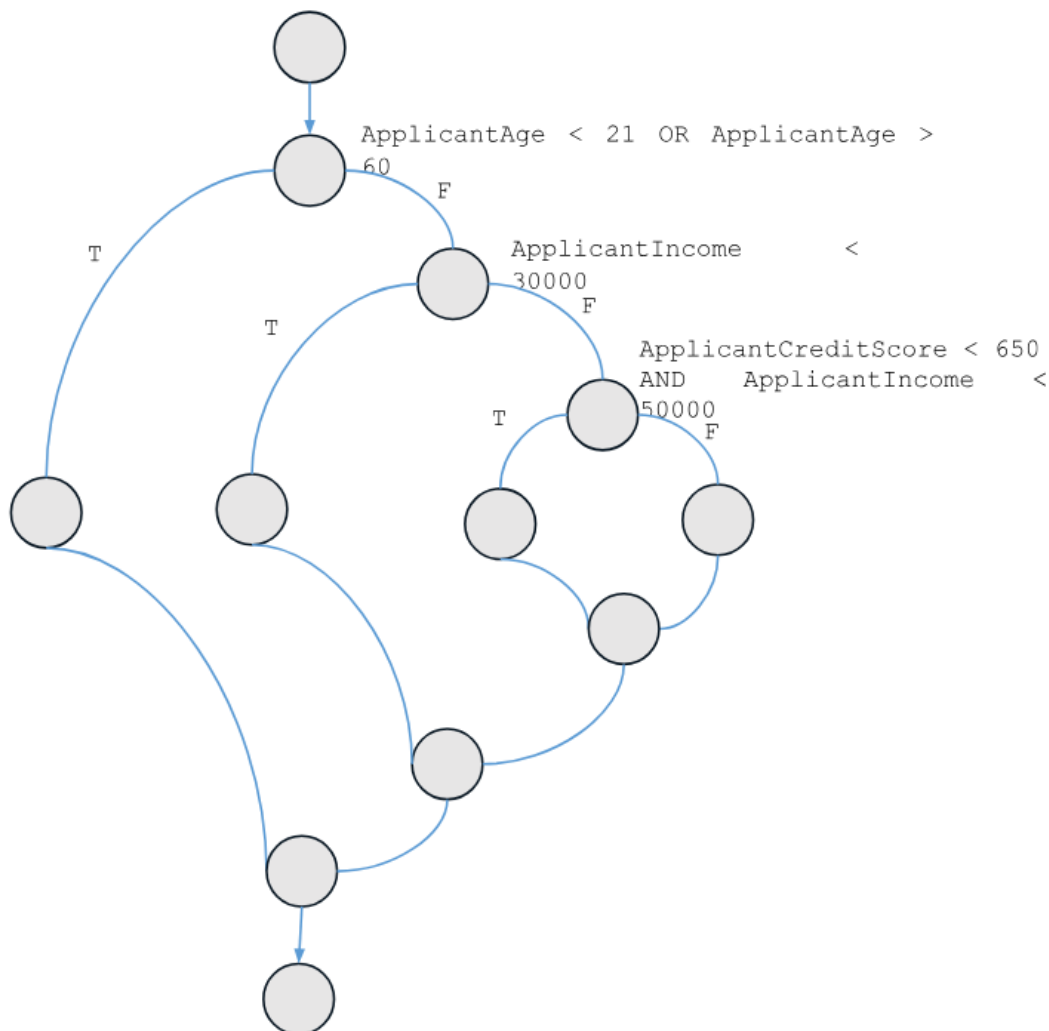
- А) Да се конструира модел за тестване с граф на управляващия поток, от който след това да се дефинират тестови сценарии за структурно тестване (тестване по метода на бялата кутия).
- Б) Да се опишат основните стъпки при конструиране на модела, като се посочат номерата на редовете, които се вземат предвид.
- В) Колко процента е покритието на условните оператори при следния тестов сценарий:
ApplicantAge=20, ApplicantIncome=45000, ApplicantCreditScore=900?
- Г) Да се дефинират тестови сценарии за постигане на 100% покритие на изразите.
- Д) Да се дефинират тестови сценарии за постигане на 100% покритие на условните оператори.

Критерии за оценяване

- Модел за тестване с граф на управляващия поток: 3,5 т.
- Описани стъпки за конструиране на модела: 2,5 т.
- Определен процент на покритие на примерния тестов сценарий: 1 т.
- Дефинирани тестови сценарии за 100% покритие на изразите: 2,5 т.
- Дефинирани тестови сценарии за 100% покритие на условните оператори: 2,5 т.
- Точките за задачата се закръгляват до цяло число.

Примерно решение

А)



Б) Последователност на конструиране на граф на управляващия поток.

- Асоцииране на обработващите възли с изразите за присвояване, извикване на процедури или функции
- Асоцииране на възлите за взимане на решение с изразите за условен преход „if-then-else“ или „if-then“
- Създаване на специален тип възли за разклонение и асоциирането им с изразите за цикъл

- Асоцииране на началния и крайния възел на графа с първия и последния израз в програмата

В) Отговор: 33%

Г) Очаква се дефиниране на 4 тестови сценарии за постигане на 100% покритие на изразите.

Reading inputs (lines 9, 10, 11)
Each if condition (lines 13, 17, 21)
Each print statement (lines 15, 19, 23, 25)

Test Case	ApplicantAge	ApplicantIncome	ApplicantCreditScore	Expected Output	Reason
TC1	20	60000	700	"Applicant not eligible due to age"	Age rejection (line 15)
TC2	25	25000	700	"Applicant not eligible due to income"	Income rejection (line 19)
TC3	30	40000	600	"Applicant not eligible due to low credit score and income"	Credit & income rejection (line 23)
TC4	35	60000	700	"Applicant eligible for loan"	Eligible print (line 25)

Д) Очаква се дефиниране на 4 тестови сценарии за постигане на 100% покритие на условните оператори.

Условни оператори:

D1: ApplicantAge < 21 OR ApplicantAge > 60
D2: ApplicantIncome < 30000
D3: ApplicantCreditScore < 650 AND ApplicantIncome < 50000

Покритие на всички разклонения:

3a D1: true and false
3a D2: true and false (only executed if D1 is false)
3a D3: true and false (only executed if D1 and D2 are false)

Test Case	ApplicantAge	ApplicantIncome	ApplicantCreditScore	Explanation
TC1	20	40000	700	D1 true (age < 21), stops after D1 true
TC2	30	25000	700	D1 false, D2 true (income < 30000), stops
TC3	35	40000	600	D1 false, D2 false, D3 true (low credit+income)
TC4	40	60000	700	D1 false, D2 false, D3 false (eligible)

Задача 5.

Нека V е линейно пространство над полето на реалните числа $\mathbb{R}$ и нека V има базис e_1, e_2 и e_3 . Нека φ е линеен оператор, който в посочения базис има матрица

$$A = \begin{pmatrix} 0 & 1 & -1 \\ 6 & 1 & 2 \\ 6 & -2 & 5 \end{pmatrix}.$$

- а) Да се намери базис, в който матрицата на φ е диагонална, както и матрицата на оператора в този базис.
- б) Да се докаже, че подпространството U на V , където $U = \{a \in V \mid \varphi(a) = 3a\}$, е φ -инвариантно подпространство на V .

Примерно решение

- а) Първо ще намерим характеристичните корени на φ . За целта конструираме характеристичния полином:

$$f_A(\lambda) = \det(A - \lambda E) = \begin{vmatrix} -\lambda & 1 & -1 \\ 6 & 1 - \lambda & 2 \\ 6 & -2 & 5 - \lambda \end{vmatrix} = -\lambda(\lambda - 3)^2.$$

Характеристичните корени са $\lambda_1 = 0$ и $\lambda_{2,3} = 3$ – цели числа и значи принадлежат на полето $\mathbb{R}$, т. е. те са и собствени стойности на φ .

Търсим базис от собствени вектори на оператора φ , отговарящи на намерените собствени стойности.

- 1) Нека $\lambda_1 = 0$. Намираме ФСР на системата с матрица $A - 0.E$. Получаваме

$$A = \begin{pmatrix} 0 & 1 & -1 \\ 6 & 1 & 2 \\ 6 & -2 & 5 \end{pmatrix} \sim \begin{pmatrix} 0 & 1 & -1 \\ 2 & 0 & 1 \\ 6 & -2 & 5 \end{pmatrix}.$$

Векторът $v_1 = (-1, 2, 2)$ е собствен, отговарящ на собствената стойност $\lambda_1 = 0$.

- 2) Търсим собствените вектори за $\lambda_{2,3} = 3$. От теорията знаем, че те са линейно независими с v_1 . Получаваме ХСЛУ с матрица

$$A - 3E = \begin{pmatrix} -3 & 1 & -1 \\ 6 & -2 & 2 \\ 6 & -2 & 2 \end{pmatrix} \sim \begin{pmatrix} -3 & 1 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Намираме ФСР на системата и получаваме собствените вектори $v_2 = (-1, 0, 3)$ и $v_3 = (1, 3, 0)$.

В базиса от собствени вектори v_1, v_2 и v_3 операторът φ има диагонална матрица $D = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 3 \end{pmatrix}$.

- б) Очевидно пространството $\mathbb{U}$ се състои от собствените вектори, отговарящи на собствена стойност 3 и нулевия вектор. Тогава $\mathbb{U} = l(v_2, v_3)$ и $\varphi(v_{2,3}) = 3v_{2,3}$. Ако $u = \alpha_2 v_2 + \alpha_3 v_3 \in \mathbb{U}$, то

$$\varphi(u) = \alpha_2 \varphi(v_2) + \alpha_3 \varphi(v_3) = 3(\alpha_2 v_2 + \alpha_3 v_3) \in \mathbb{U}.$$

Окончателно получихме, че $\mathbb{U}$ е φ -инвариантно подпространство на $\mathbb{V}$.

Критерии за оценяване

- а) При пълно и вярно решение се присъждат 10 т.

За намиране на собствените стойности – 4 точки. За намиране на собствения вектор v_1 , отговарящ на собствена стойност 0 – 2 точки. За намиране на собствените вектори v_2 и v_3 (намиране на ФСР) – 3 точки. За експлицитно записване на диагоналната матрица на оператора в базиса от собствени вектори – 1 точка.

- б) При пълно и вярно решение се присъждат 2 т.

Задача 6.

Да се пресметне интегралът

$$\int_0^1 x \operatorname{arctg}(x) \, dx.$$

Примерно решение и критерии за оценяване

Преобразуваме последователно:

$$\begin{aligned}
 \int_0^1 x \operatorname{arctg}(x) \, dx &= \\
 (1 \text{ т.}) &= \frac{1}{2} \int_0^1 \operatorname{arctg}(x) \, d(x^2) = \\
 (3 \text{ т.}) &\quad (\text{интегрираме по части}) &= \frac{1}{2} [x^2 \operatorname{arctg}(x)] \Big|_0^1 - \frac{1}{2} \int_0^1 x^2 \, d \operatorname{arctg}(x) = \\
 (1 \text{ т.} + 1 \text{ т.}) &= \frac{1}{2} [1^2 \cdot \operatorname{arctg}(1) - 0] - \frac{1}{2} \int_0^1 x^2 \frac{1}{x^2 + 1} \, dx = \\
 (1 \text{ т.} + 1 \text{ т.}) &= \frac{\pi}{8} - \frac{1}{2} \int_0^1 \frac{x^2 + 1 - 1}{x^2 + 1} \, dx = \\
 (2 \text{ т.}) &= \frac{\pi}{8} - \frac{1}{2} [x - \operatorname{arctg}(x)] \Big|_0^1 = \\
 (1 \text{ т.}) &= \frac{\pi}{8} - \frac{1}{2} [(1 - 0) - (\operatorname{arctg}(1) - \operatorname{arctg}(0))] = \\
 (1 \text{ т.}) &= \frac{\pi}{4} - \frac{1}{2}
 \end{aligned}$$

Чернова